GUIA DE APOIO • GITHUB DESKTOP

CURSO	UNIDADE CURRICULAR
LICENCIATURA EM ENGENHARIA INFORMÁTICA	TECNOLOGIAS DA INTERNET I
DOCENTE	
MÁRIO AMORIM	

GITHUB DESKTOP: CRIAR E PUBLICAR UM REPOSITÓRIO

Passo a passo para guardares as práticas e os trabalhos de TINT1 no GitHub e os manteres actualizados. No TP02, em grupo, a equipa usa o mesmo repositório, com issues, um quadro Kanban e branches (secções 6 a 10).

> **Nota:** as capturas foram feitas com um repositório de exemplo (`Exercicios Aula` / `cesae-python-exercicios`). No teu caso aparecem o nome da tua pasta e do teu repositório (por exemplo, `tint1-praticas`).

1. INSTALAR E CONFIGURAR (UMA VEZ)

1. Instalar o **Git**: descarregar em **git-scm.com** e instalar com as opções por omissão.
2. Instalar o **GitHub Desktop**: descarregar em **desktop.github.com**.
3. Confirmar que o Git ficou bem instalado, no terminal:

```
> git --version
git version 2.x...
```

4. Dizer ao Git quem és (aparece em cada commit). Usa o teu nome e o **email da tua conta GitHub**:

```
> git config --global user.name "Nome Apelido"
> git config --global user.email "o-teu-email@exemplo.com"
```

2. Adicionar a pasta no GitHub Desktop

1. No GitHub Desktop, clica em **Add an Existing Repository from your local drive...**

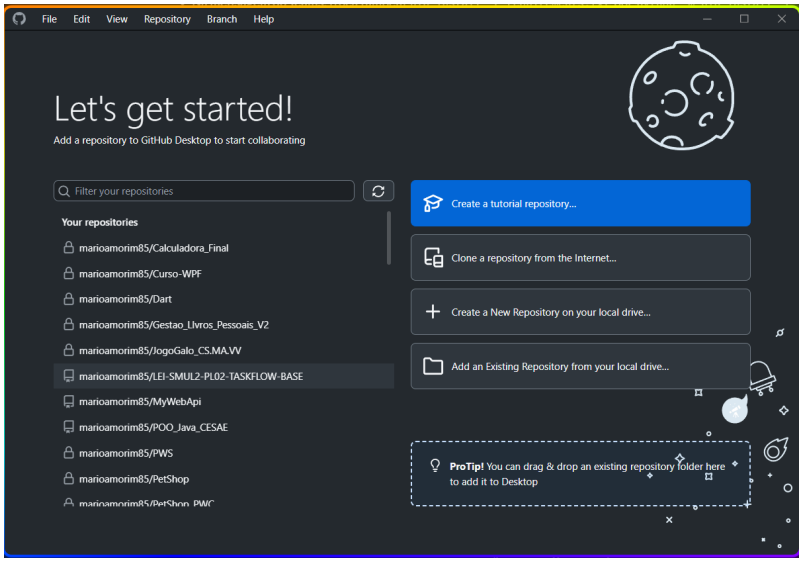


Figura 1 – Ecrã inicial do GitHub Desktop.

2. Escolhe a pasta onde guardas as práticas de TINT1, por exemplo (com as pastas `pl01`, `pl02`, `pl03` lá dentro):

```
...\Documentos\TINT1
```

3. Como a pasta ainda não é um repositório Git, aparece uma mensagem como «This directory does not appear to be a Git repository». Clica em **create a repository**.

3. Criar o repositório

Preenche os dados:

Campo	Valor
Name	deixa como está (o nome da pasta)
Description	Práticas de Tecnologias da Internet I - ISTEC Porto
Local path	a pasta das práticas
Git ignore	None (para HTML, CSS e JavaScript não é preciso)
License	None

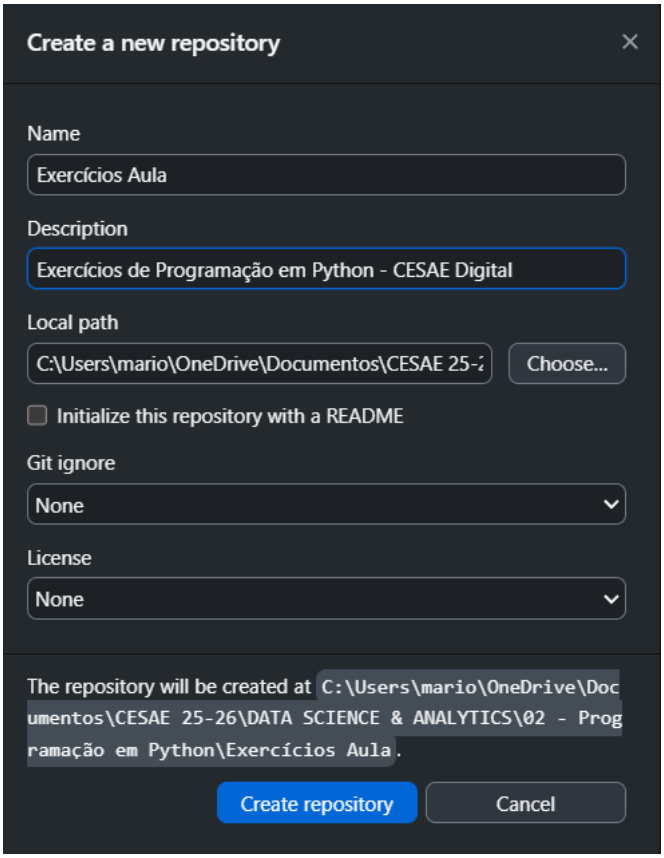


Figura 2 – Criar o repositório.

1. Clica em **Create repository**.

4. Publicar no GitHub

1. Clica em **Publish repository**.

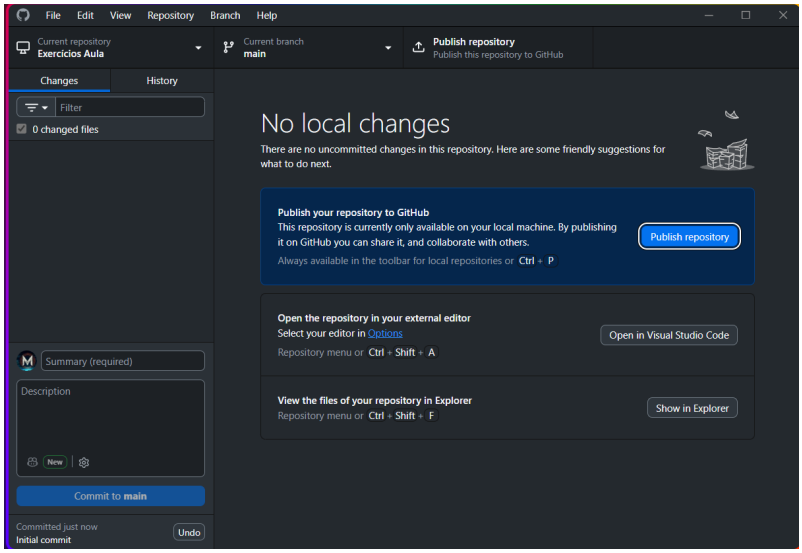


Figura 3 – Publicar o repositório no GitHub.

2. Muda o nome para algo **sem acentos e sem espaços**, por exemplo:

```
tint1-praticas
```

É mais limpo para o GitHub, para os links e para os comandos Git.

3. Decide se queres **privado ou público**. Deixa «Keep this code private» marcado se for só para ti.
4. Clica em **Publish repository**. No teu perfil (`github.com/o-teu-utilizador?tab=repositories`) já vês o repositório publicado.

5. Guardar alterações (commit e push)

1. Abre a pasta no **VS Code**. Ao alterares os ficheiros, as alterações aparecem em **Changes** (tanto no VS Code como no GitHub Desktop).

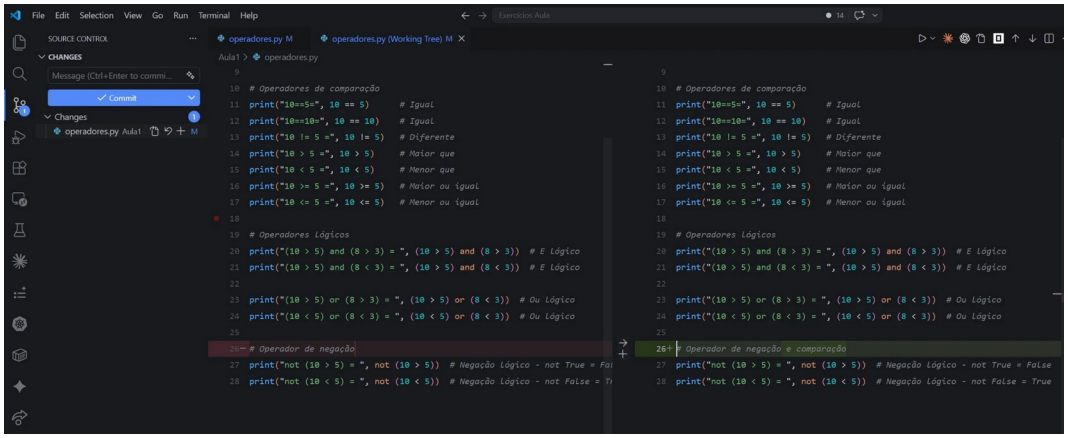


Figura 4 – As alterações aparecem em Changes, no VS Code.

2. Escreve uma **mensagem de commit** (no campo Summary) e clica em **Commit to main**.

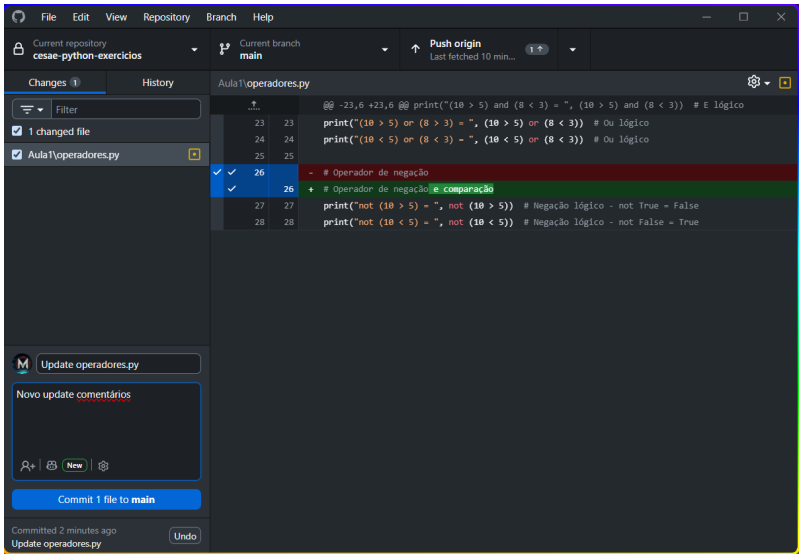


Figura 5 – Mensagem de commit e Commit to main.

3. Clica em **Push origin** para enviar as alterações. O repositório no github.com fica actualizado.

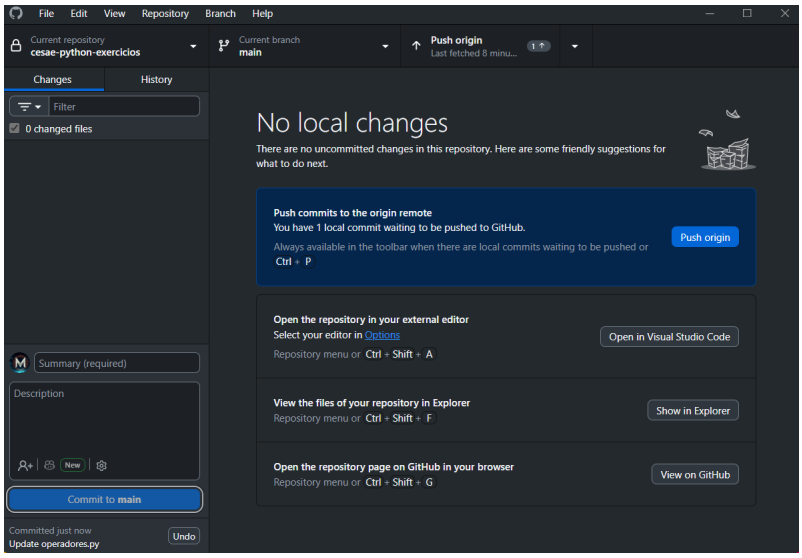


Figura 6 – Push origin envia os commits para o GitHub.

Boa prática: um commit por tarefa concluída, com uma mensagem que diga o que fizeste (por exemplo, «PL03: formulário de inscrição»), e não «alterações».

6. Adicionar colaboradores (trabalho em equipa)

No TP02 trabalham em equipa no **mesmo repositório**. O dono do repositório convida os colegas para poderem fazer commit e push.

1. No repositório em **github.com**, abre **Settings** e, em Access, clica em **Collaborators**. Depois clica em **Add people**.

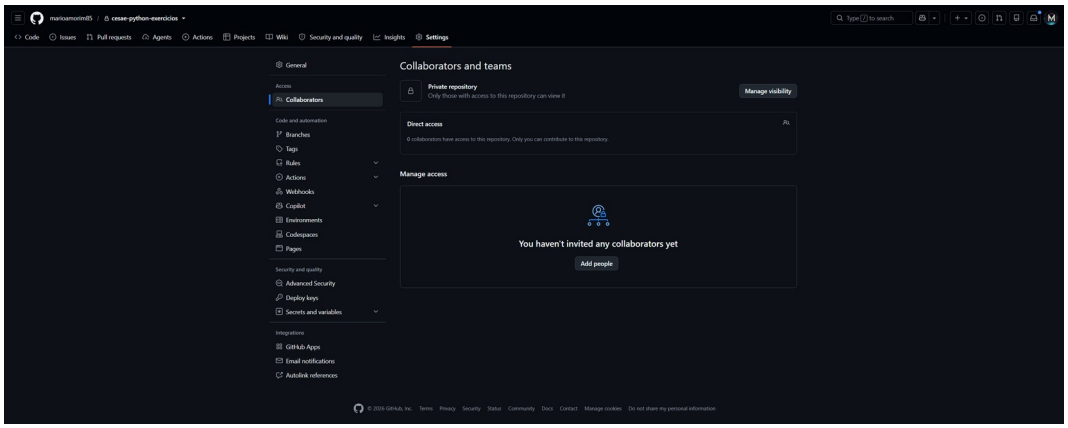


Figura 7 – Settings › Collaborators › Add people.

2. Procura cada colega pelo **username, nome ou email da conta GitHub** e clica em **Add to repository**.

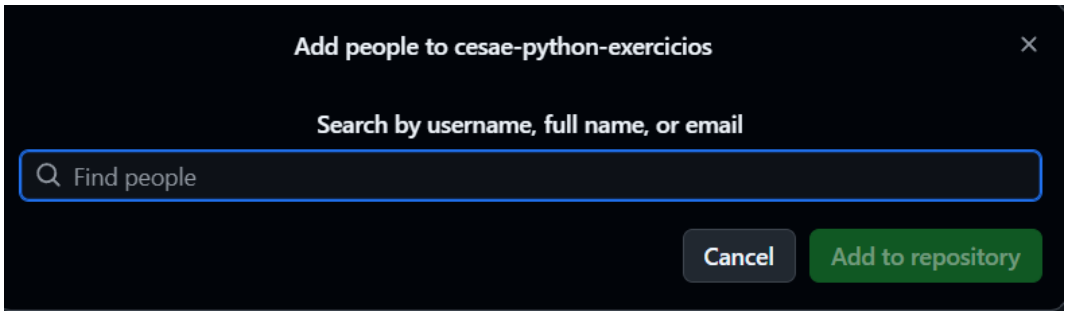


Figura 8 – Convidar um colega.

Cada colega recebe um **convite** (por email ou em `github.com/notifications`) que tem de **aceitar**. Para começar a trabalhar, abre o GitHub Desktop em **File → Clone repository** e escolhe o repositório da equipa.

7. Issues: registar as tarefas

As **issues** são a lista de tarefas do projecto. Cada tarefa fica registada, com responsável e categoria.

1. No repositório, abre o separador **Issues** e clica em **New issue**.

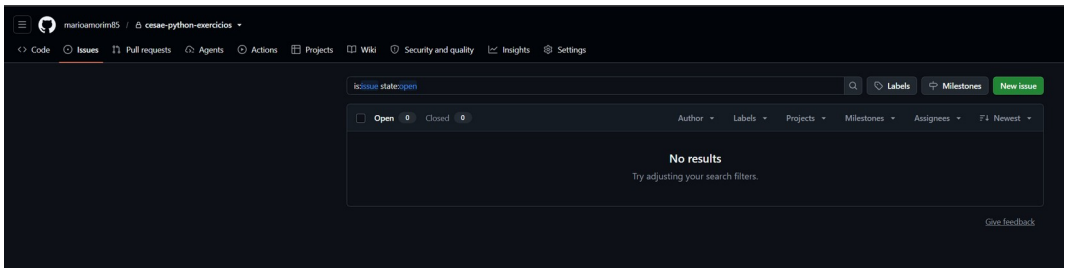


Figura 9 – Separador Issues.

2. Preenche o **Título** e a **Descrição**. À direita podes já atribuir a um colega (**Assignees**), associar ao **Project** e pôr etiquetas (**Labels**). Por fim, clica em **Create**.

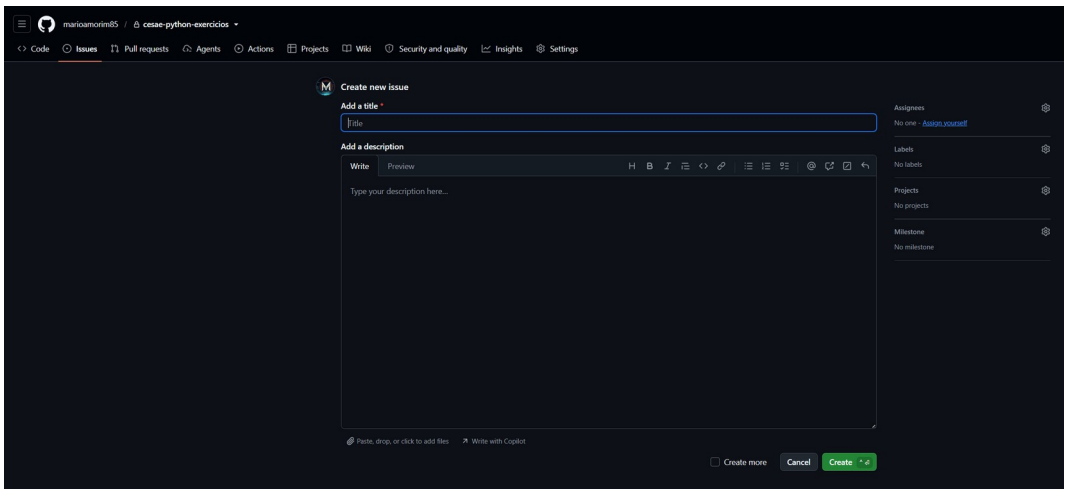


Figura 10 – Criar uma issue.

Etiquetas (**Labels**) úteis: html, css, javascript, design, documentação, apresentação.

Exemplos de tarefas para o TP02:

Tarefa	Responsável	Label
Estrutura HTML das páginas	Ana	html
Folha de estilos e layout responsivo	Bruno	css
Formulário de contacto com validação	Carla	html
Interactividade e Web Storage	Bruno	javascript
Validar HTML e CSS no W3C	Ana	documentação
Preparar a apresentação	Carla	apresentação

Dica: podes fechar uma issue automaticamente escrevendo `Closes #3` na mensagem de commit.

8. Project (Kanban): acompanhar o trabalho

O quadro **Kanban** mostra as tarefas por estado e ajuda a equipa a ver o que falta fazer.

1. No repositório, abre o separador **Projects** e clica em **New project**.

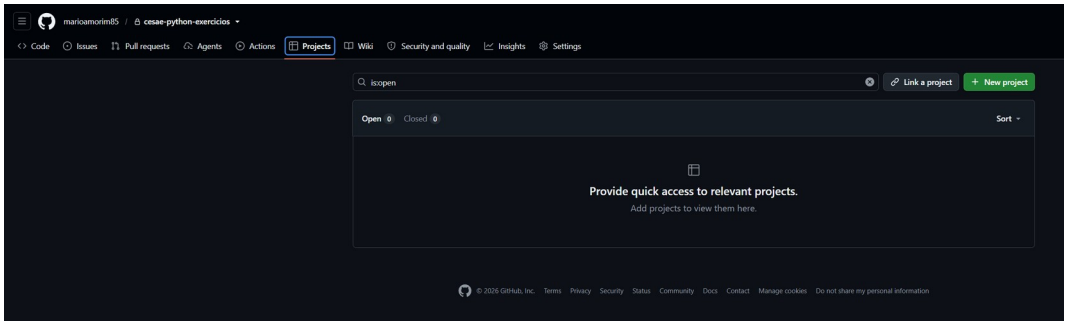


Figura 11 – Separador Projects.

2. Escolhe o modelo **Kanban**.

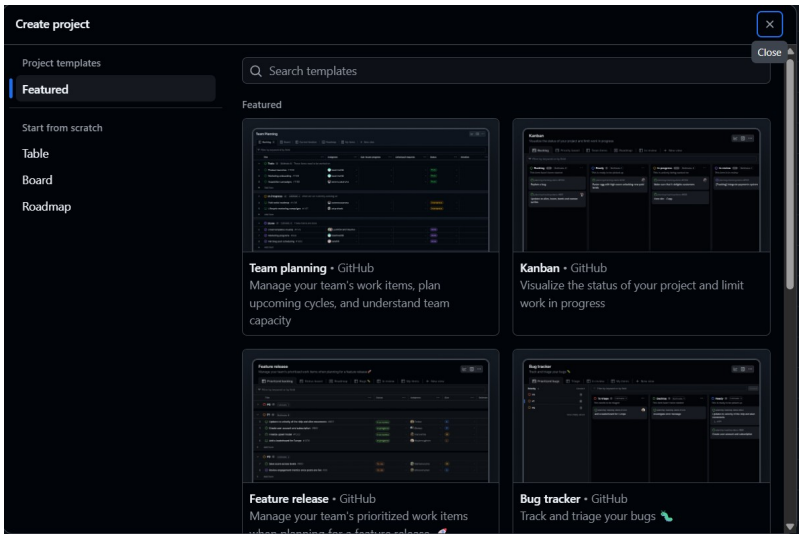


Figura 12 – Modelos de projecto.

3. Confirma a importação das issues do repositório e clica em **Create project**.

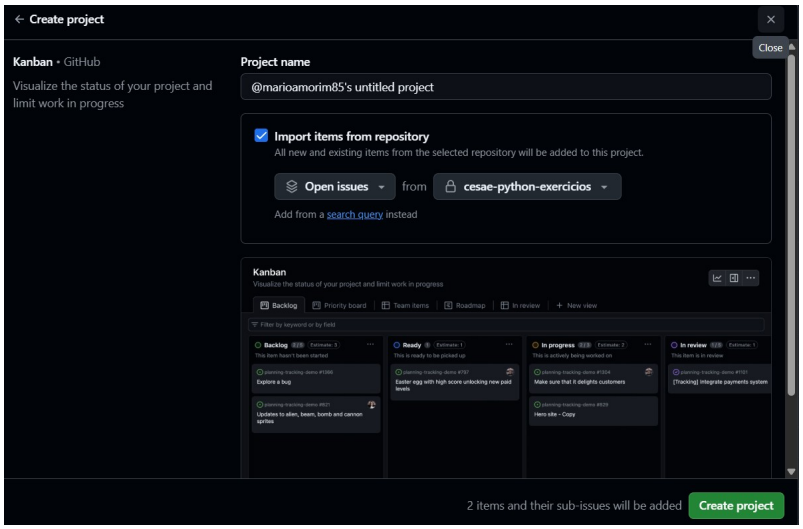


Figura 13 – Importar as issues e criar o projecto.

4. As tarefas aparecem no **Backlog**. Vai **arrastando** cada cartão entre as colunas (Backlog → Ready → In progress → In review → Done) à medida que trabalham.

5. Para mudar o nome do projecto, clica no ícone do **lápis** ao lado do título.

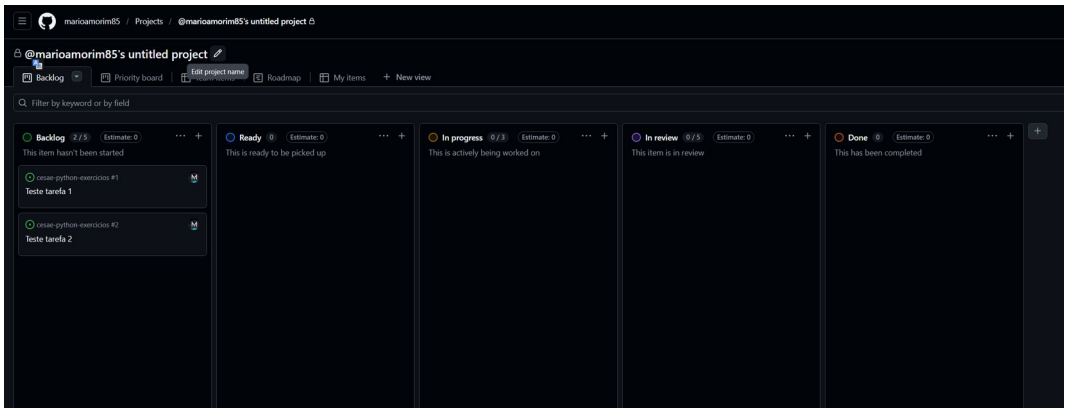


Figura 14 – Quadro Kanban com as tarefas.

9. Sincronizar antes de trabalhar (Pull origin)

Quando vários trabalham no mesmo repositório, **antes de começares** e antes de cada push deves trazer para o teu computador o que os colegas já enviaram. Assim evitas conflitos.

1. No GitHub Desktop, clica em **Fetch origin** (canto superior direito) para verificar se há novidades no GitHub.
2. Se houver commits novos, o botão passa a **Pull origin** (com uma seta para baixo e o número de commits). Clica nele para os trazer.

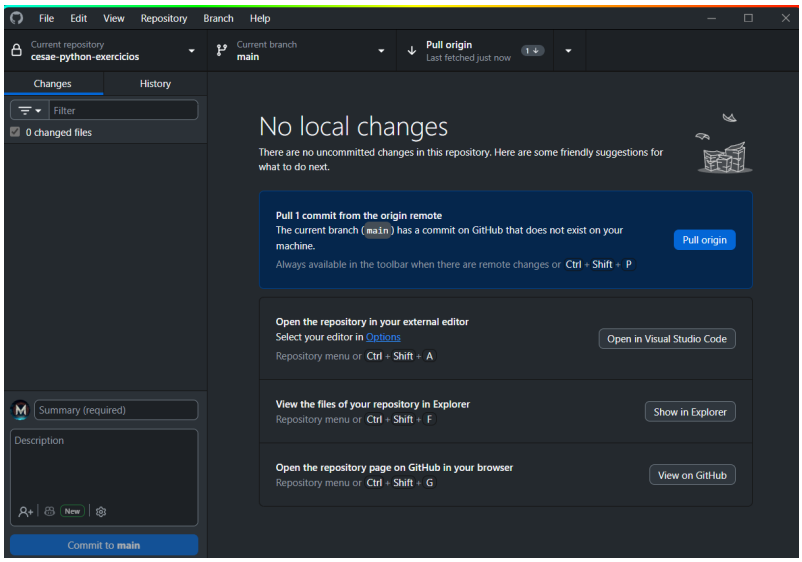


Figura 15 – Pull origin traz os commits dos colegas.

Regra de ouro: Pull origin → trabalhar → Commit → Pull origin outra vez → Push origin.

10. Branches e Pull Requests (opcional, modo de equipa)

Em equipa, o ideal é cada pessoa trabalhar numa **branch** (uma cópia paralela) e, no fim, propor a junção ao `main` através de um **Pull Request**, que os colegas podem rever antes de aceitar. Assim o `main` mantém-se sempre estável.

1. No GitHub Desktop, clica em **Current branch** e depois em **New branch**.

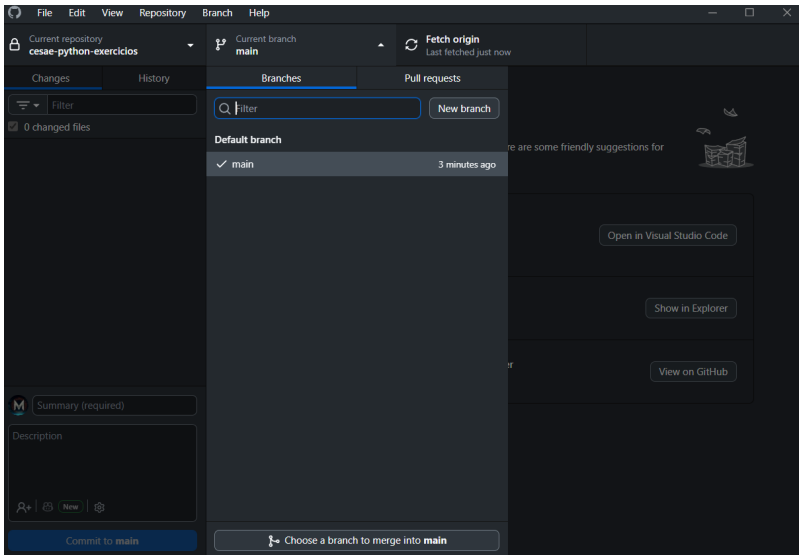


Figura 16 – Current branch › New branch.

2. Dá um nome descritivo (ex.: `pagina-contactos`) e clica em **Create branch**. Passas a trabalhar nessa branch, sem mexer no `main`.

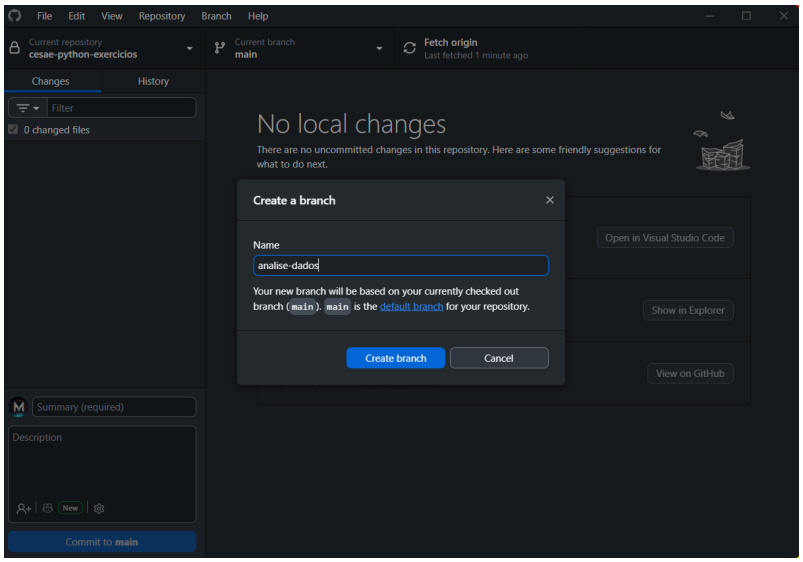


Figura 17 – Criar a branch.

3. Faz as alterações, **Commit**, e clica em **Publish branch**. A seguir clica em **Create Pull Request** (abre o GitHub no browser).

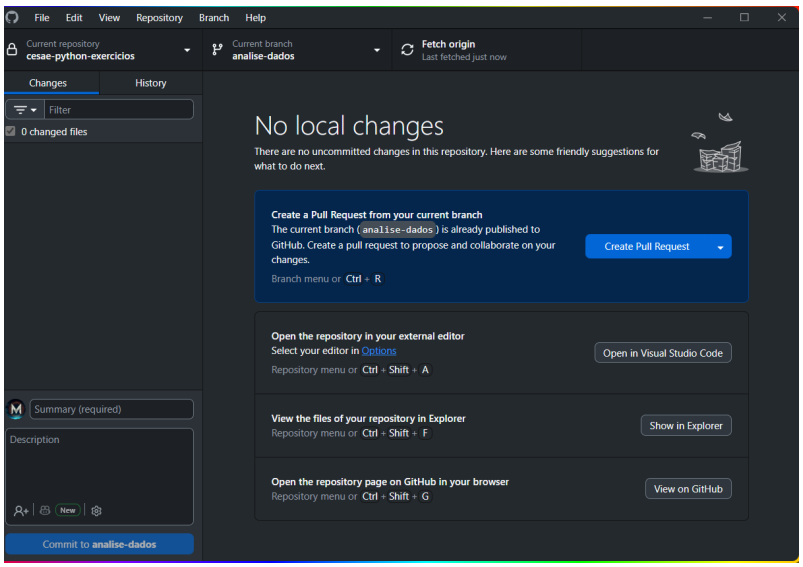


Figura 18 – Publicar a branch e criar o Pull Request.

4. No GitHub, confirma o **título** e a descrição e clica em **Create pull request**.

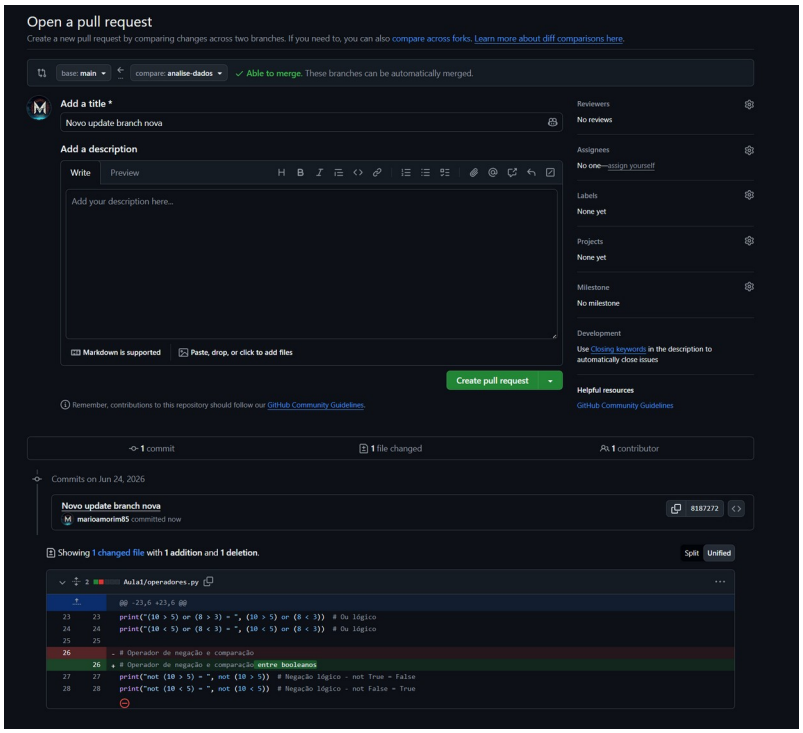


Figura 19 – Abrir o Pull Request no GitHub.

5. Se estiver tudo bem, aparece «No conflicts with base branch». Um colega revê e clica em **Merge pull request** para juntar ao `main`.

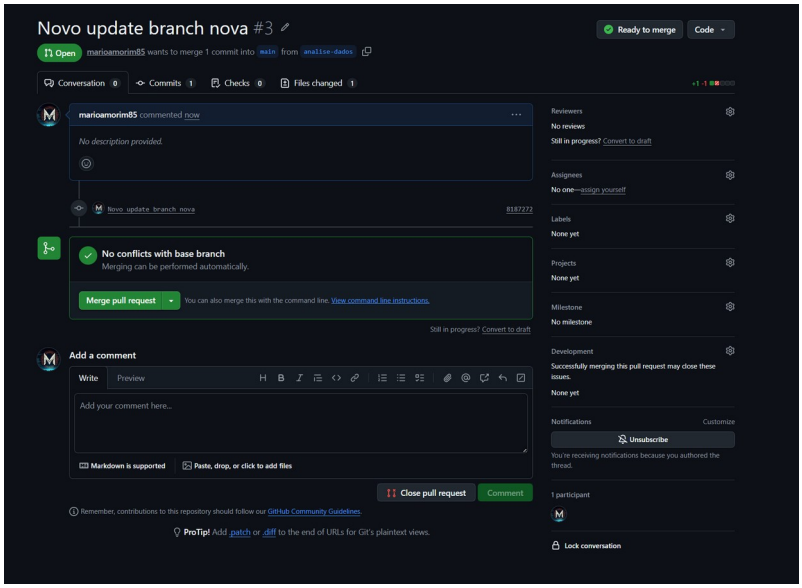


Figura 20 – Sem conflitos: Merge pull request.

Depois de juntar: volta ao `main` (**Current branch → main**) e faz **Pull origin** para actualizar o teu computador.

11. (Opcional) Remover o Git de uma pasta

Se quiseres desfazer o repositório, no terminal (PowerShell):

```
> cd "...\Documentos\TINT1"
> Remove-Item -Recurse -Force .git
```

E apaga o repositório no GitHub em **Settings → Danger Zone → Delete this repository**.

Atenção: apagar a pasta `.git` remove todo o histórico de commits dessa pasta. Os teus ficheiros ficam intactos.